

Jira Task: Create Category

Role: brand

Authentication: Required

POST /brands/:brandId/categories

Success: 201

شرح كامل – إزاي تنفذ التاسك ده في مشروع ThreadMarket

الخلاصة في سطر واحد: البراند لوحده يقدر يعمل قسم (Category) جديد مرتبط بيه، بشرط إن اسم القسم ميتكررش عنده. بس.

0. المواصفات المطلوبة من Jira

العنصر	التفاصيل
HTTP Method	POST
Endpoint	/api/v1/brands/{brandId}/categories
Headers	Authorization: Bearer <accessToken>
Body	name (إجباري) – imageUrl (اختياري)
Validation	اسم القسم لازم يكون فريد ضمن نفس البراند
Authentication	مطلوب (توكن)
Roles المصرح بيها	brand فقط
Success Response	201 – يرجع id بس

مثال على ال Request Body

```
{
  "name": "Women's Wear",
  "imageUrl": "https://cdn.example.com/category.png"
}
```

ال Response الناجح – 201

```
{
  "success": true,
  "message": "Category created successfully",
  "data": {
    "id": "uuid"
  }
}
```

ال Error Responses

الموقف	Status	السبب
Validation Error	400	ال body غلط – زي اسم فاضي أو format غلط
Forbidden	403	المستخدم مش براند، أو البراند مش بتاعه
Not Found	404	البراند نفسه مش موجود
Conflict	409	اسم القسم متكرر ضمن نفس البراند
Internal Server Error	500	أي خطأ غير متوقع

1. المطلوب منك كودياً

محتاج تعمل موديول جديد اسمه `category` على نفس نمط الموديولات الموجودة
(`brand` و `user`):

```
src/modules/category/  
├─ category.model.ts  
├─ category.service.ts  
├─ category.controller.ts  
├─ category.validators.ts  
└─ category.routes.ts
```

- **model** – كلام مع قاعدة البيانات عبر Prisma (إنشاء القسم).
- **service** – منطق الأعمال: فحص إن البراند موجود، فحص إن المستخدم هو صاحب البراند، فحص تكرار الاسم.
- **controller** – بياخد ال `request`، بينادي ال `service`، ويرجع ال `201` بال `id`.
- **validators** – Zod للتحقق من ال `body`.
- **routes** – يربط كل ده بال `(auth + authorize + validate)` middleware.

نقطة مهمة: ال endpoint الحالي في `server.ts` هو `/api/v1/brands` (سطر 25). الراوت الجديد `/brands/:brandId/categories` لازم يبقى تحت ال `brands` — يعني إما تضيفه جوه `brand.routes.ts` أو تعمل router جديد وتركّبه برضه تحت `/api/v1/brands`.

2. قاعدة البيانات – إزاي السكيما شايلة الفكرة؟

شوف ال `Category` الموجود في `prisma/schema.prisma` (سطر 241):

```
model Category {  
  id          String      @id @default(uuid()) @db.Uuid  
  name        String      @unique @map("category_name")  
  media       Json
```

```

ishidden Boolean @default(false)
Products Product[]
createdAt DateTime @default(now())
updatedAt DateTime @default(now())
}

```

المشكلة: الموديل ده مافيوش **brandId** – يعني القسم مش مرتبط بأي براند،
والـ **@unique** على الاسم حاليًا على مستوى كل النظام (global) مش على مستوى
البراند. التاسك عايز:

- كل قسم يتربط ببراند واحد (علاقة `Brand → Category`).
- الاسم يكون فريد **ضمن البراند بس** مش على مستوى العالم كله.

اللي محتاج تعمله في السكيما

```

model Category {
  id          String    @id @default(uuid()) @db.Uuid
  brandId     String    @map("brand_id") @db.Uuid
  name        String    @map("category_name")
  imageUrl    String?   @map("image_url")
  ishidden    Boolean    @default(false)
  Products    Product[]
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @default(now())

  brand Brand @relation(fields: [brandId], references: [id],
onDelete: Cascade)

  @@unique([brandId, name]) // الاسم فريد ضمن البراند ←
  @@map("categories")
}

```

- شلنا **@unique** من الاسم وحطيناه كـ **@@unique([brandId, name])**.
- أضفنا **brandId** والعلاقة **brand**.

• حوّلنا media (اللي كانت Json) لـ imageUrl String? عشان تناسب الحقل imageUrl المطلوب في التاسك.

وخذ بالك: كمان محتاج تضيف categories Category[] في موديل Brand عشان العلاقة تكتمل من الطرفين.

3. الأخطاء اللي حصلت لما عدّلت السكيما – وإزاي اتصلحت

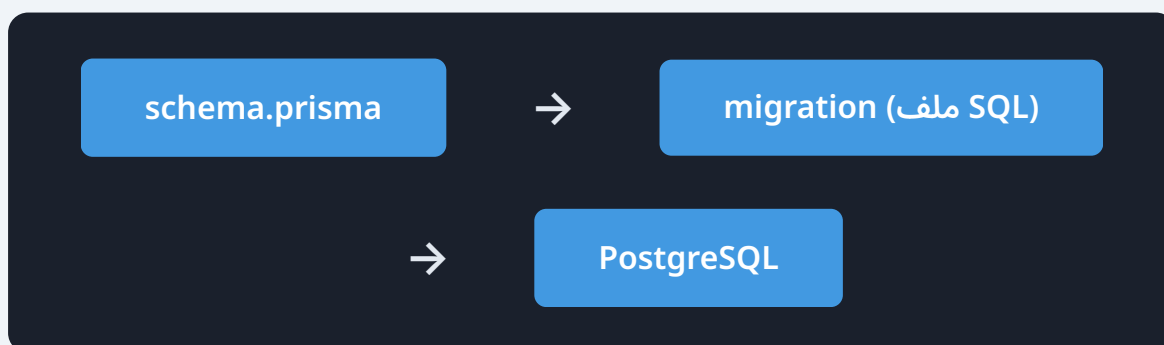
بعد ما كتبت التعديل، عملت npx prisma validate فطلع 5 أخطاء. الاتنين الأساسيين:

#	الخطأ	السبب	الحل
1	Type "Cascade" is neither a built-in type... خطأ في سطر)	كتبت onDelete Cascade من غير نقطتين، ف Prisma مش فاهمة إنها attribute	تحوّل لـ onDelete: Cascade
2	missing an opposite relation field on the model "Brand"	علاقة Category → Brand كانت "واحد بيه" بس، والناحية الثانية (فاضية مفيش فيها إشارة للـ Category)	ضفنا في موديل Brand : categories Category[]

قاعدة مهمة في Prisma: أي علاقة لازم تكون معرّفة من الطرفين – الطرف الأول (صاحب الـ FK) والطرف الثاني (الناحية اللي عندها الـ array). لو واحد بس ناقص، الـ validate هيفشل.

4. إزاي تحفظ التعديلات دي في قاعدة البيانات؟ (Migrations)

التعديل على `schema.prisma` لوحده مش كفاية — ده بس "الرسم". عشان الرسم ده يتحول لجدول فعلاً في قاعدة البيانات، محتاج **migration**. يعني ال migration بيعمل:



سلسلة الأوامر كاملة (من الأهم للأهم):

```
npx prisma validate      # 1) تأكد إن السكيما سليمة (معملناه وعدّي)
npx prisma migrate dev   # 2) للتغييرات + يعمل push ويعمل migration يعمل
generate
npx prisma generate      # 3) أحياناً ال Prisma Client يعمل تحديث للـ
migrate (بينفذا لوحده)
npx prisma migrate status # مطبّق ومين لأ migration يشوف مين (اختياري)
```

ملاحظة على `migrate dev`: هيسألك تسمّي ال migration — اكتب اسم وصفي بال English زي `add_brand_relation_to_categories`. وكمان هو بيرجعك `generate` أوتوماتيك، بس لو حسيت أن ال Prisma Client لسه مش عارف ال fields الجديدة، شغل `generate` يدوي.

5. شرح العلاقة العكسية (Reverse Relation) بالتفصيل

علاقة ال Category بال Brand هي علاقة **One-to-Many**: براند واحد → كذا قسم. وأي علاقة كده ليها **ناحيتين** في Prisma:

الناحية	فين	السبب
<code>brand Brand</code> <code>@relation(...)</code>	في <code>Category</code>	دي الناحية "المفصلة" – فيها الـ <code>brandId</code> الفعلي (ForeignKey) اللي بيتخزن في جدول الـ <code>categories</code>
<code>categories</code> <code>Category[]</code>	في Brand	دي الناحية "المعكسة" – مش بتخزن حاجة فعلاً، دي بس بتقول Prisma إنه من البراند تقدر توصل لأقسامه: <code>prisma.brand.findUnique({ include: { categories: true } })</code>

لماذا لازم الناحيتين؟ عشان Prisma تبقى عارفة إن الحقلين دول بيربطوا بعض. من غير الناحية الثانية بتطلع: `missing an opposite relation field`. زي علاقة بالعقل: لو قلت "الوالد عنده أبناء" ولازم تذكر إن "الابن عنده أب" – الاثنين يوصفوا نفس العلاقة.

لاحظ كمان الـ `onDelete: Cascade`: يعني لو البراند اتشال من قاعدة البيانات، كل أقسامه هتتشال معاه أوتوماتيك – مفيش أقسام يتيمة.

هل `categories Category[]` معناها علاقة One-to-Many؟ — أيوه

أيوه، بالضبط. العلاقة one-to-many بتتكون من **ناحيتين** في Prisma، ودي القاعدة البسيطة:

الناحية	الكود	المعنى
الطرف "الواحد" (Brand)	<code>categories</code> <code>Category[]</code>	البراند الواحد → كذا قسم

الطرف "المتعدد" (Category)	brand Brand	كل قسم → براند واحد
-------------------------------	-------------	---------------------

- **اللي عنده []** = الطرف "الواحد" في العلاقة (بيحوي كذا عنصر).
- **اللي من غير []** = الطرف "المتعدد" (كل عنصر ييمتلك واحد بس).
- في قاعدة البيانات، المفتاح الأجنبي `brandId` بيتخزن في **جدول الـ categories** (طرف many) مش في جدول الـ `brands`.
- مثال واقعي: براند "Zara" عنده [رجالي , حريمي , إكسسوارات] ، `categories` ، وكل قسم من دول بيشاور على Zara بحقل `brand` . دي علاقة one-to-many كاملة.

6. خطوات التنفيذ بالترتيب

1. **عَدِّل السكيمات:** أضف `brandId` + علاقة `brand` في `Category` و `categories` في `Brand` ، واعمل `@@unique([brandId, name])` . ثم `npx prisma migrate dev`
2. **حضر الـ validator:** باستخدام `zod` — `brandId` من الـ `params` لازم `uuid` ، والـ `body` فيه `name` إجباري (string غير فاضي) و `imageUrl` اختياري (URL صالح).
3. **اعمل الـ model:** طريقة `createCategory(brandId, name, imageUrl)` تستدعي `prisma.category.create` .
4. **اعمل الـ service:**
 - تأكد إن البراند موجود (إلا 404).
 - تأكد إن `req.user` هو صاحب البراند (إلا 403).
 - اعمل `findFirst` على `{ brandId, name }` — لو لقيته، اعمل `throw 409` "Category name already exists"
 - أنشئ القسم وارجع `id` .

5. **اعمل ال controller:** استخدم `asyncHandler` زي باقي المشروع، ورجع
- ```
res.status(201).json({ success: true, message: "Category created successfully", data: { id } })
```
6. **اعمل ال routes:** `POST /:brandId/categories` مع `authenticate` → `controller` → `validate(schema)` → `authorize("brand")`
7. **رَكِّب الراوت:** في `server.ts` تحت `/api/v1/brands`

## 7. الأدوات اللي في المشروع هتساعدك

| الأداة                           | المكان                                             | بتعمل إيه                                                                   |
|----------------------------------|----------------------------------------------------|-----------------------------------------------------------------------------|
| <code>authenticate</code>        | <code>src/shared/middleware/authenticate.ts</code> | بيحط <code>req.user</code> (فيه <code>id</code> و <code>role</code> )       |
| <code>authorize(...roles)</code> | <code>src/shared/middleware/authorize.ts</code>    | بيرفض غير البراند بـ 403                                                    |
| <code>validate(schema)</code>    | <code>src/shared/middleware/validate.ts</code>     | بيتحقق من ال <code>params</code> وال <code>body</code> بـ <code>zod</code>  |
| <code>asyncHandler</code>        | <code>src/shared/utils/asyncHandler.ts</code>      | بيلفّ على ال <code>async</code> ويبعت الأخطاء لل <code>error handler</code> |
| <code>AppError</code>            | <code>src/shared/errors/AppError.ts</code>         | عشان ترمي أخطاء بال <code>status code</code> الصح (400/403/404/409)         |
| <code>prisma</code>              |                                                    |                                                                             |

الكلينت للتعامل مع قاعدة  
البيانات

src/config/  
database.ts

## 8. نمط كود مرجعي من الموجود (نفس الستايل)

ده نفس النمط بتاع brand.routes.ts — طَبِّقه على ال routes الجديدة:

```
router.post("/:brandId/categories",
 authenticate, // 1) مين الداخل
 authorize("brand"), // 2) بس البراند
 validate(createCategorySchema), // 3) تحقق من المدخلات
 createCategory, // 4) المنطق
);
```

## 9. سيناريوهات الاختبار

| # | السيناريو                            | النتيجة المتوقعة                            |
|---|--------------------------------------|---------------------------------------------|
| 1 | POST بدون توكن                       | 401 Unauthorized                            |
| 2 | توكن بس المستخدم role = user         | 403 Forbidden                               |
| 3 | براند بيضيف قسم اسمه موجود عنده      | 409 Conflict                                |
| 4 | براند بيضيف اسم موجود عند براند ثاني | 201 Created (لأن الفريدة على مستوى البراند) |
| 5 | براند brandId مش موجود               | 404 Not Found                               |
| 6 | POST من غير name في ال body          | 400 Validation Error                        |

|                                   |   |                           |
|-----------------------------------|---|---------------------------|
| براند صاحب البراند بيسوي اسم جديد | 7 | data.id بيرجع 201 Created |
|-----------------------------------|---|---------------------------|

## 10. ملاحظات أخيرة

**قبل ما تسلم:** اعمل `npm run build` (typecheck) عشان تتأكد مفيش أخطاء TypeScript، وجرب الـ endpoint في Postman بعد ما تسجل دخول ببراند فعلاً.

**المراجع داخل المشروع:** موديول `user` و `brand` دول أفضل مثال على الستايل المطلوب (validators → model → service → controller → routes).